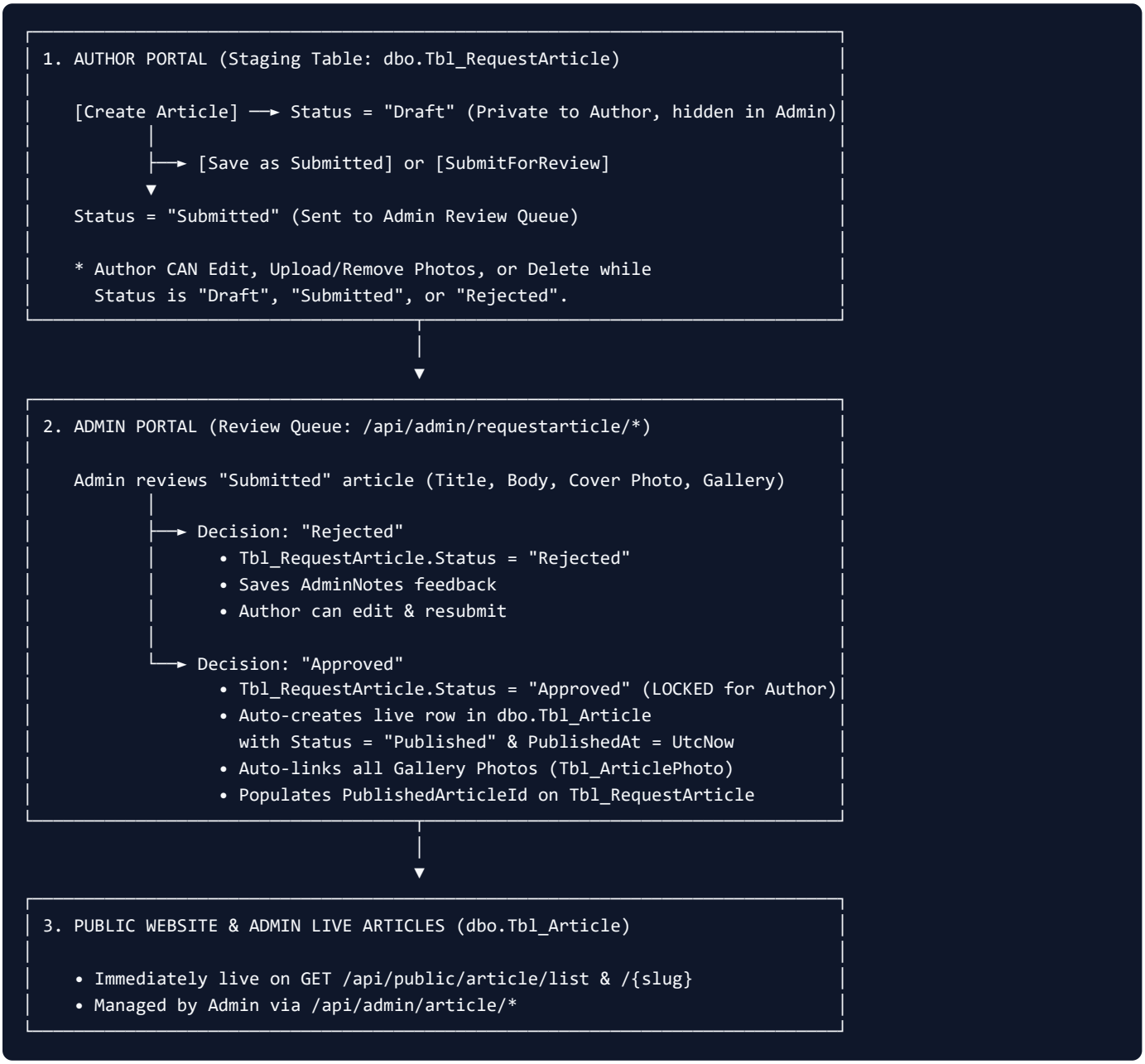
Frontend Developer Guide: Author-to-Admin Article Approval & Publishing Workflow

This document is the complete Frontend reference for the new **Article Staging, Admin Review, and Publishing Workflow** across:

- **Author Portal API** (`MagizineAuthor.Api` — `/api/author/article/*`)
- **Admin Portal API** (`MagizineAdmin.Api` — `/api/admin/requestarticle/` & *`/api/admin/article/`*)
- **Public Magazine API** (`MagizinePublic.Api` — `/api/public/article/*`)

---

1. Workflow Overview & Status Lifecycle



---

2. What Changed (Frontend Migration Summary)

Area	Old Behavior	New Behavior (What Frontend Must Update)
Author Primary ID	Used `ArticleId` from `Tbl_Article`	Uses **`RequestArticleId`** (26-char ULID) from `Tbl_RequestArticle`
Author Create / Update	Wrote directly to `Tbl_Article`	Writes to `Tbl_RequestArticle` with `Status = "Draft"` or `"Submitted"` (or `"Submit"`)
Author Delete	Did not exist	Added **`POST /api/author/article/Delete`** (`{ "RequestArticleId": "..." }`)
Author Lock Rule	Locked when `Status != "Draft"`	Editable & deletable until Admin approves (**locked only when `Status === "Approved"`**)
Admin Review Queue	Pitch fields (`PitcherName`, `ProposedTitle`, `PitchBody`)	Full article fields (`Title`, `SubTitle`, `Slug`, `Body`, `PhotoUrl`, `CategoryName`, `AuthorName`, `Photos`, `PublishedArticleId`)
Admin Draft Visibility	Admin could see drafts	Author `"Draft"` rows are **strictly hidden** from `/api/admin/requestarticle/List` & `Detail`
Admin Approval Action	Only changed pitch status	Atomically creates `Published` article in `Tbl_Article`, links gallery photos, and returns `PublishedArticleId`
Public Pitch Endpoint	`POST /api/public/requestarticle/submit`	**Removed** (only authenticated Authors submit articles)

3. UI State & Permission Matrix

3.1 Author Portal UI Rules (`Tbl_RequestArticle`)

`Status` Value	Badge Color Suggestion	Can Author Edit?	Can Author Delete?	Can Author Submit?	Visible to Admin?	Notes for Frontend UI
`"Draft"`	Gray / Neutral	✅ Yes	✅ Yes	✅ Yes	❌ No (Private)	Show **"Edit"**, **"Submit for Review"**, and **"Delete"** buttons.
`"Submitted"`	Blue / Amber	✅ Yes	✅ Yes	✅ (Resubmit)	✅ Yes	Show **"Under Review"** badge. Author can still edit or delete before Admin approves.
`"Rejected"`	Red	✅ Yes	✅ Yes	✅ Yes	✅ Yes	Display `AdminNotes` alert box so Author sees why it was rejected, can edit, and click **"Resubmit"**.
`"Approved"`	Green	❌ **Locked**	❌ **Locked**	❌ **Locked**	✅ Yes	Disable/hide Edit & Delete buttons. Show **"Published"** badge and link to live article using `PublishedArticleId` or `Slug`.

4. Author Portal API Reference (`MagizineAuthor.Api`)

Base Route: `/api/author/article`

4.1 Create Article (`POST /api/author/article/Create`)

- **Content-Type**: `multipart/form-data`
- **Description**: Creates a new article in `Tbl_RequestArticle` with `Status = "Draft"` or `"Submitted"`.

`FormData` Fields:

Field Name	Type	Required	Description
`Title`	`string`	✅ Yes	Headline (max 250 chars)
`SubTitle`	`string`	No	Subtitle (max 500 chars)
`Body`	`string`	No	Rich text / HTML article body
`ArticleCategoryId`	`string`	✅ Yes	26-char ULID of category
`Status`	`string`	No	`"Draft"` (default) or `"Submitted"` (also accepts `"Submit"`)
`CoverPhoto`	`File`	No	Cover image file (`.jpg`, `.jpeg`, `.png`, `.webp`, max 5 MB)
`PhotoUrl`	`string`	No	Fallback external URL if `CoverPhoto` file is not uploaded
`PhotoCaption`	`string`	No	Cover photo caption (max 200 chars)
`PhotoCredit`	`string`	No	Cover photo credit (max 100 chars)
`GalleryItems[i].File`	`File`	No	Gallery image file (`i = 0, 1, ...`, max 10 items)
`GalleryItems[i].Caption`	`string`	No	Caption for gallery photo `i`
`GalleryItems[i].Credit`	`string`	No	Credit for gallery photo `i`
`GalleryItems[i].SortOrder`	`number`	No	Display order (`1, 2, 3...`) for gallery photo `i`

TypeScript Example:

```
export async function createAuthorArticle(
  baseUrl: string,
  form: {
    title: string;
    subTitle?: string;
    body?: string;
    articleCategoryId: string;
    status: "Draft" | "Submitted";
    coverPhoto?: File | null;
    photoUrl?: string;
    photoCaption?: string;
    photoCredit?: string;
    galleryItems?: Array<{
      file: File;
      caption?: string;
      credit?: string;
      sortOrder: number;
    }>;
  },
  token?: string
) {
  const formData = new FormData();
  formData.append("Title", form.title);
  if (form.subTitle) formData.append("SubTitle", form.subTitle);
  if (form.body) formData.append("Body", form.body);
  formData.append("ArticleCategoryId", form.articleCategoryId);
  formData.append("Status", form.status);

  if (form.coverPhoto) {
    formData.append("CoverPhoto", form.coverPhoto);
  } else if (form.photoUrl) {
    formData.append("PhotoUrl", form.photoUrl);
  }
  if (form.photoCaption) formData.append("PhotoCaption", form.photoCaption);
  if (form.photoCredit) formData.append("PhotoCredit", form.photoCredit);

  form.galleryItems?.forEach((item, i) => {
    formData.append(`GalleryItems[${i}].File`, item.file);
    formData.append(`GalleryItems[${i}].Caption`, item.caption ?? "");
    formData.append(`GalleryItems[${i}].Credit`, item.credit ?? "");
    formData.append(`GalleryItems[${i}].SortOrder`, String(item.sortOrder));
  });

  const res = await fetch(`${baseUrl}/api/author/article/Create`, {
    method: "POST",
    headers: token ? { Authorization: `Bearer ${token}` } : undefined,
    body: formData,
  });
  return await res.json();
}
```

4.2 Update Article (`POST /api/author/article/Update`)

- **Content-Type**: `multipart/form-data`
- **Description**: Updates an existing article in `Tbl_RequestArticle`. Allowed while `Status !== "Approved"`.

`FormData` **Fields:**

Field Name	Type	Required	Description
`RequestArticleId`	`string`	✅ Yes	26-char ULID of the article in `Tbl_RequestArticle`
`Title`	`string`	✅ Yes	Headline
`SubTitle`	`string`	No	Subtitle
`Body`	`string`	No	Rich text / HTML body
`ArticleCategoryId`	`string`	✅ Yes	26-char ULID of category
`Status`	`string`	No	`"Draft"` or `"Submitted"` (also accepts `"Submit"`)
`CoverPhoto`	`File`	No	New cover image file (if replacing cover photo)
`PhotoUrl`	`string`	No	Existing `PhotoUrl` (to keep current cover photo when `CoverPhoto` is not uploaded)
`PhotoCaption`	`string`	No	Cover photo caption
`PhotoCredit`	`string`	No	Cover photo credit
`NewGalleryItems[i].File`	`File`	No	New gallery image file to append
`NewGalleryItems[i].Caption`	`string`	No	Caption for new gallery photo `i`
`NewGalleryItems[i].Credit`	`string`	No	Credit for new gallery photo `i`
`NewGalleryItems[i].SortOrder`	`number`	No	Sort order for new gallery photo `i`
`RemoveArticlePhotoIds[i]`	`string`	No	26-char `ArticlePhotoId` ULIDs of existing gallery photos to delete

---

4.3 Submit Draft for Review (`POST /api/author/article/SubmitForReview`)

- **Content-Type**: `application/json`
- **Description**: Changes status to `"Submitted"` and sets `SubmittedAt` timestamp.

Request Body:

```
{
  "RequestArticleId": "01JREQ0000000000000000002"
}
```

4.4 Delete My Article (`POST /api/author/article/Delete`)

- **Content-Type**: `application/json`
- **Description**: Soft-deletes the Author's article and its staged gallery photos. Blocked if `Status === "Approved"`.

Request Body:

```
{
  "RequestArticleId": "01JREQ00000000000000000002"
}
```

4.5 Get My Article Detail (`POST /api/author/article/Detail`)

- **Content-Type**: `application/json`

Request Body:

```
{
  "RequestArticleId": "01JREQ00000000000000000001"
}
```

Response Body (`Result<RequestArticleResModel>`):

```
{
  "IsSuccess": true,
  "IsError": false,
  "RespType": "Success",
  "RespCode": "MS#000",
  "RespDesp": "Success",
  "RespDespParameter": null,
  "Data": {
    "RequestArticleId": "01JREQ00000000000000000001",
    "Title": "Monastic Architecture in Contemporary Homes",
    "SubTitle": "How Cistercian simplicity inspires serene urban residences",
    "Slug": "monastic-architecture-in-contemporary-homes",
    "Body": "<p>An essay examining how contemporary architects...</p>",
    "PhotoUrl": "/uploads/articles/2026/10/01JCOVER00000000000000001.webp",
    "PhotoCaption": "Minimalist limestone courtyard in Kyoto.",
    "PhotoCredit": "Photo by Kenji Sato",
    "ArticleCategoryId": "01JCAT00000000000000000003",
    "CategoryName": "Musing",
    "AuthorId": "01JAUTHOR0000000000000001",
    "AuthorName": "Elena Rostova",
    "Status": "Submitted",
    "SubmittedAt": "2026-10-11T01:22:41.1765137+06:30",
    "ReviewedAt": null,
    "ReviewedByAdminId": null,
    "AdminNotes": null,
    "PublishedArticleId": null,
    "CreatedAt": "2026-10-11T01:22:41.1765137+06:30",
    "UpdatedAt": null,
    "Photos": [
      {
        "ArticlePhotoId": "01JAPHT000000000000000003",
        "ArticleId": null,
        "RequestArticleId": "01JREQ00000000000000000001",
        "PhotoUrl": "https://images.unsplash.com/photo-1600585154526-990dced4db0d",
        "Caption": "Raw concrete and warm oak timber framing.",
        "Credit": "Photo by Kenji Sato",
        "SortOrder": 1,
        "CreatedAt": "2026-10-11T01:22:41.1765137+06:30"
      }
    ]
  }
}
```

4.6 Get My Article List (`POST /api/author/article/List`)

- **Content-Type**: `application/json`

Request Body:

```
{
  "Page": 1,
  "PageSize": 20,
  "SearchKeyword": null,
  "ArticleCategoryId": null,
  "Status": null
}
```

> `Status` filter can be `null` (all my articles), `"Draft"`, `"Submitted"`, `"Approved"`, or `"Rejected"`.

---

5. Admin Portal Review Queue API Reference (`MagizineAdmin.Api`)

Base Route: `/api/admin/requestarticle`

5.1 List Submitted Article Requests (`POST /api/admin/requestarticle/List`)

- **Content-Type**: `application/json`
- **Description**: Returns paginated list of Author article requests. **Author `"Draft"` rows are automatically excluded.**

Request Body:

```
{
  "Page": 1,
  "PageSize": 20,
  "SearchKeyword": null,
  "ArticleCategoryId": null,
  "AuthorId": null,
  "Status": "Submitted"
}
```

> Pass `"Status": "Submitted"` for the pending review queue, or `"Status": null` to see all `"Submitted"`, `"Approved"`, and `"Rejected"` requests.

5.2 Get Request Article Detail (`POST /api/admin/requestarticle/Detail`)

- **Content-Type**: `application/json`
- **Description**: Returns the full article content and its `Photos` gallery so the Admin can preview the entire article before approving.

Request Body:

```
{
  "RequestArticleId": "01JREQ00000000000000000001"
}
```

5.3 Review Request Article — Approve or Reject (`POST /api/admin/requestarticle/Review`)

- **Content-Type**: `application/json`
- **Description**: Reviews a `"Submitted"` article request.
- When `Status` is `"Approved"` :
- Automatically inserts a new row into `dbo.Tbl_Article` with `Status = "Published"` and `PublishedAt = UtcNow` .
- Applies `IsFeatured` and `IsSpotlight` flags to the new `Tbl_Article` .
- Links all gallery photos in `dbo.Tbl_ArticlePhoto` (`ArticleId = newArticleId`) without duplicating files.
- Sets `Tbl_RequestArticle.Status = "Approved"` (locking it from Author edits) and sets `PublishedArticleId = newArticleId` .
- When `Status` is `"Rejected"` :
- Sets `Tbl_RequestArticle.Status = "Rejected"` and saves `AdminNotes` .

Request Body (Approve Example):

```
{
  "RequestArticleId": "01JREQ00000000000000000001",
  "Status": "Approved",
  "AdminNotes": "Excellent architecture essay. Approved for publication.",
  "IsFeatured": true,
  "IsSpotlight": false
}
```

Request Body (Reject Example):

```
{
  "RequestArticleId": "01JREQ00000000000000000001",
  "Status": "Rejected",
  "AdminNotes": "Please add higher-resolution courtyard photography and expand the conclusion.",
  "IsFeatured": false,
  "IsSpotlight": false
}
```

Response Body (After Approval):

```
{
  "IsSuccess": true,
  "IsError": false,
  "RespType": "Success",
  "RespCode": "MS#000",
  "RespDesp": "Success",
  "Data": {
    "RequestArticleId": "01JREQ00000000000000000001",
    "Title": "Monastic Architecture in Contemporary Homes",
    "Slug": "monastic-architecture-in-contemporary-homes",
    "Status": "Approved",
    "SubmittedAt": "2026-10-11T01:22:41.1765137+06:30",
    "ReviewedAt": "2026-10-11T03:15:00.0000000+00:00",
    "ReviewedByAdminId": "01JADMIN00000000000000001",
    "AdminNotes": "Excellent architecture essay. Approved for publication.",
    "PublishedArticleId": "01JART99999999999999999999",
    "Photos": [
      {
        "ArticlePhotoId": "01JAPHT000000000000000003",
        "ArticleId": "01JART99999999999999999999",
        "RequestArticleId": "01JREQ00000000000000000001",
        "PhotoUrl": "https://images.unsplash.com/photo-1600585154526-990dced4db0d",
        "Caption": "Raw concrete and warm oak timber framing.",
        "Credit": "Photo by Kenji Sato",
        "SortOrder": 1
      }
    ]
  }
}
```

6. TypeScript Interfaces for Frontend

```
export type RequestArticleStatus = "Draft" | "Submitted" | "Approved" | "Rejected";

export interface ArticlePhotoDto {
  ArticlePhotoId: string;
  ArticleId?: string | null;
  RequestArticleId?: string | null;
  PhotoUrl: string;
  Caption?: string | null;
  Credit?: string | null;
  SortOrder: number;
  CreatedAt: string;
}

export interface RequestArticleDto {
  RequestArticleId: string;
  Title: string;
  SubTitle?: string | null;
  Slug: string;
  Body?: string | null;
  PhotoUrl?: string | null;
  PhotoCaption?: string | null;
  PhotoCredit?: string | null;
  ArticleCategoryId: string;
  CategoryName: string;
  AuthorId: string;
  AuthorName: string;
  Status: RequestArticleStatus;
  SubmittedAt?: string | null;
  ReviewedAt?: string | null;
  ReviewedByAdminId?: string | null;
  AdminNotes?: string | null;
  PublishedArticleId?: string | null;
  CreatedAt: string;
  UpdatedAt?: string | null;
  Photos: ArticlePhotoDto[];
}

export interface ReviewRequestArticlePayload {
  RequestArticleId: string;
  Status: "Approved" | "Rejected";
  AdminNotes?: string | null;
  IsFeatured: boolean;
  IsSpotlight: boolean;
}
```